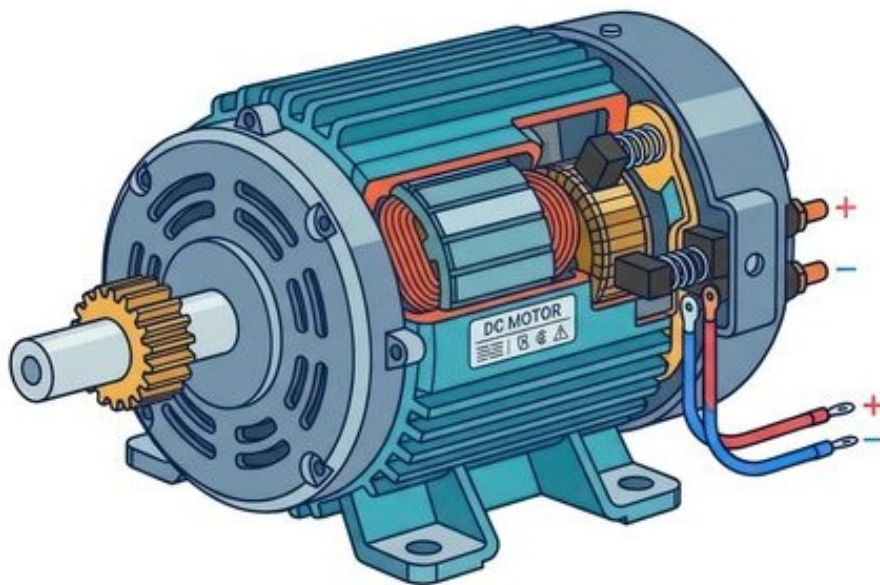


Motor Isolation & Protection Fundamentals

Task 2.6 – Relay vs. Optocoupler vs. SSR, ESP32 Protection, and EMI



*Dana Natsheh
Cyber Robot
IoT*

1. Executive Summary

This report is a research-and-reference deliverable rather than a build log: it does not describe a new working prototype, but instead consolidates the theory that should sit underneath any project that puts an ESP32 in direct or near-direct control of a brushed DC motor through a driver such as the L298N. Three questions frame the report. First, when a project needs to switch something – a signal, a low-current logic line, or the motor’s own high-current supply – which of a relay, an optocoupler, or a solid-state relay (SSR) is actually the right component, and why are the three routinely confused despite doing fundamentally different jobs (Section 2)? Second, once that switching device is chosen, what concretely protects the ESP32 itself from the electrical environment a motor creates – flyback diodes, optocoupler isolation boards, and a separate power rail – and how do those three protections address three different failure modes rather than one (Section 3)? Third, at the modest scale of a single brushed motor and one or two relays, which electromagnetic-interference (EMI) mitigations actually matter, as opposed to the much larger catalog of techniques used in industrial motor-drive installations (Section 4)? Section 5 closes with a consolidated checklist tying all three topics together for the next time a motor is wired into an ESP32-based project.

2. Relay vs. Optocoupler vs. SSR

2.1 What Each Component Actually Does

The three components are frequently listed side by side as if they were interchangeable “switches,” but they solve two different problems: isolating a signal, and switching power. Conflating the two is the single most common source of a burned-out GPIO or an underpowered motor connection.

An optocoupler is a signal isolator, not a power switch. It pairs an LED with a phototransistor (or photodiode) inside one package, so that a control-side current through the LED is converted to light and picked back up on the output side with no direct electrical connection between the two halves. Because the output-side transistor can only pass a small current – typically tens of milliamps – an optocoupler cannot drive a motor directly; its job is to let a low-voltage, low-current logic signal cross a galvanic isolation boundary safely, so that whatever is on the other side of that boundary can never feed a fault current back into the ESP32’s GPIO.



Figure 1: The PC817 optocoupler

A mechanical relay is a power switch built from a coil and a set of physical contacts. A small control current through the coil creates a magnetic field that pulls a mechanical armature, closing (or opening) a separate, physically isolated set of contacts that can carry a much larger current – commonly a few amps for a small signal relay used in a project like this one. Because the switching element is a physical contact, a relay can pass current in either direction and switch AC or DC equally well, but the moving parts wear out over repeated cycles and switching itself is relatively slow (milliseconds).



Figure 2: A common 5V single-channel relay module, the type referenced in Table 1

A solid-state relay (SSR) is a hybrid: an optocoupler for input isolation, driving a semiconductor power switch instead of a mechanical contact. The control-side LED triggers the isolation stage exactly as in a standalone optocoupler, but the output stage is a triac (for AC loads) or back-to-back MOSFETs (for DC loads) rather than a phototransistor, so the SSR can switch the same order of load current a mechanical relay can, but silently, with no moving parts, and with a far higher practical switching-cycle life. The combination of an optocoupler for isolation and a high-power semiconductor switch for the output stage is exactly what defines an SSR internally.



Figure 3: A real solid-state relay module (Omron G3MB-202P)

2.2 Comprehensive Comparison Table

Table 1: Comparison of relays, optocouplers, and SSRs across the properties relevant to choosing a switching device for an ESP32-plus-motor project.

Property	Mechanical Relay	Optocoupler	Solid-State Relay (SSR)
Switches power?	Yes – a physical contact carries the full load current directly	No – passes only a small signal current; cannot drive a motor on its own	Yes – a semiconductor stage (triac / MOSFETs) carries the full load current

continued on next page

Table 1 continued from previous page

Property	Mechanical Relay	Optocoupler	Solid-State Relay (SSR)
PWM-capable?	Poor fit – mechanical contacts are not rated for the switching rates PWM requires and wear out quickly under high-frequency cycling	Yes, for a logic-level signal – readily passes a PWM control signal through to the next stage	Limited – most AC SSRs are only rated for switching rates on the order of a few Hz to tens of Hz; DC-MOSFET SSRs handle PWM far better than AC-triac SSRs
Isolation type	Galvanic (physical air gap between coil and contacts)	Optical (LED/phototransistor pair; true galvanic isolation with no shared electrical path)	Optical at the input stage (identical to a standalone optocoupler), semiconductor switch at the output
Max current (typical small module)	A few amps at the contacts (e.g. common 5 V hobby relay modules are rated around 2 A to 10 A depending on part)	Tens of milliamps at the output transistor – not meant to carry load current	Comparable to a small mechanical relay at the output stage, with a continuous voltage drop across the semiconductor switch under load
ESP32-direct-drive?	No – the coil current typically exceeds a safe GPIO sourcing current and needs a transistor driver stage (the relay module’s onboard transistor, in most hobby boards)	Yes – the LED side is designed to be driven directly from a GPIO through a current-limiting resistor	Depends on the module – many SSR modules include their own input-side current limiting and can be driven directly, but this should be confirmed per datasheet rather than assumed
Switching speed / life	Slow (ms-scale); mechanical wear limits cycle life	Fast (μ s-scale is typical for the isolation stage itself)	Fast, with no mechanical wear, giving effectively unlimited switching-cycle life compared to a mechanical relay

2.3 Which One Fits Which Line on This Project

The L298N’s ENA/ENB and IN1–IN4 lines are logic-level signals, not power lines, and that is exactly what an optocoupler is built for. These pins only need to present a clean 3.3 V-compatible HIGH/LOW (or a PWM duty cycle on the enable pins) to the L298N’s onboard logic; the L298N accepts logic-level inputs compatible with common microcontroller output levels and is commonly wired directly to an ESP32’s GPIO pins. A direct GPIO-to-L298N connection works electrically, but it also means any fault on the L298N’s logic side – a shorted input, a driver-board defect, or noise coupled in from the motor side of the board – has a direct electrical path back to the ESP32. Inserting an optocoupler isolation board between the GPIO and the L298N’s inputs keeps the PWM capability those lines need (Section 2.2) while removing that direct path, which is the isolation boundary this report treats as the actual barrier protecting the ESP32 (Section 3.2).

The motor’s own main power line – the high-current path from the supply, through the L298N’s output stage, to the motor terminals – is a job for a relay or an SSR, not an optocoupler, and specifically becomes relevant wherever the project needs to cut that line entirely (an emergency stop, or a master power switch) rather than merely modulate it, since the L298N’s own H-bridge already handles direction and PWM speed control. A mechanical relay is the simpler, cheaper choice for a single on/off cut of a DC motor

supply switched infrequently (e.g. an E-stop); an SSR is worth the extra cost only if that supply line needs to be switched frequently or silently, since its semiconductor output stage avoids the contact wear a relay would accumulate under frequent cycling.

Interesting detail found during research: PWM is where the three components diverge most sharply. An optocoupler happily passes a PWM *signal* through to the next stage because it is only ever carrying a small control current. An SSR, by contrast, is switching the *load* itself, and most AC-triac SSR modules are only rated for on/off switching rates in the single-digit-to-tens-of-Hz range – datasheets for solid-state relays note that when a PWM signal is used as the input, the AC load's switching frequency needs to stay below roughly 10 Hz or the SSR's output stage cannot keep up, which rules out using a generic AC SSR as a motor speed controller. DC-output SSRs built on MOSFETs handle PWM far better, but this is exactly why the L298N's own PWM-capable H-bridge outputs, not a relay or an SSR, are what should always be doing the actual speed control in this project – the relay/SSR choice only applies to the on/off master-power question.

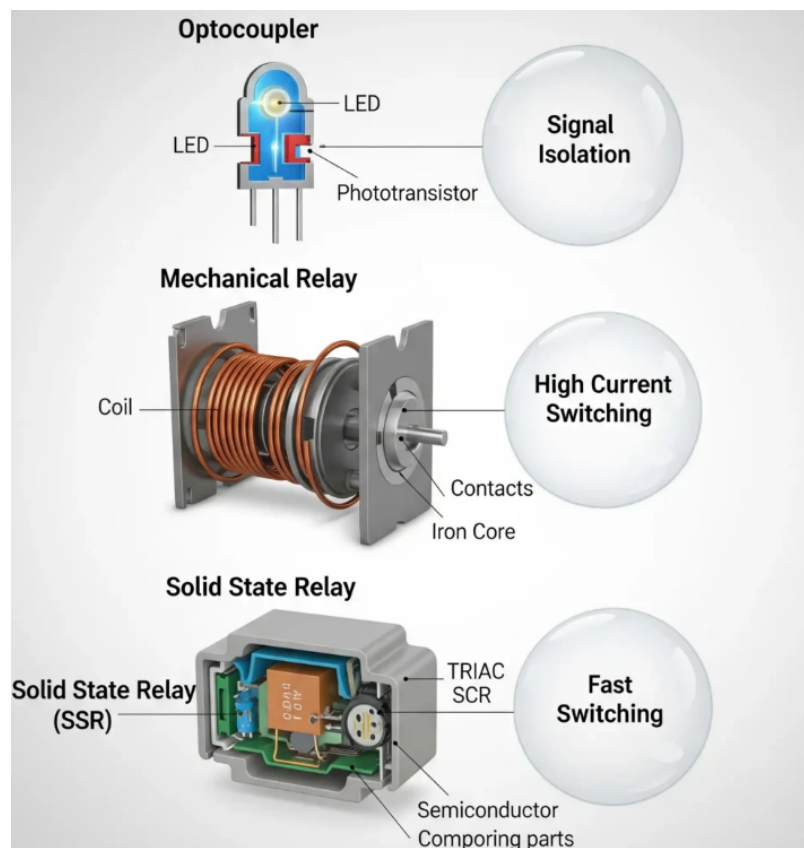


Figure 4: Internal anatomy of the three components

3. Protecting the ESP32

Section 2 established which switching device belongs on which line. This section covers the three protections that matter once those devices are in place – each addressing a distinct way the ESP32 could otherwise be damaged, not three variations on the same fix.

3.1 Flyback Diodes

A flyback diode exists because a coil or a motor cannot instantly stop conducting current when power is removed. A coil (a relay's electromagnet) or a motor's windings store energy in a magnetic field while current flows; when the driving circuit switches off, that collapsing field induces a large back-EMF voltage spike that can damage the transistor or IC that was driving the coil if nothing provides the current a safe path to keep flowing. The diode is wired in parallel with the coil or motor, in reverse polarity relative to normal operation, so it stays non-conducting (and irrelevant) while the coil is energized normally, and only turns on the instant the drive current is cut – at which point it gives the collapsing field's current a low-impedance loop to dissipate in, clamping the voltage spike instead of letting it hit the driving circuit.

Placement matters as much as presence. On a relay module, the diode belongs directly across the coil terminals, as physically close to the coil as the layout allows, because any wiring between the coil and the diode adds its own inductance, and a diode that is not close enough cannot clamp a spike fast enough to protect against it. On a DC motor, the same principle places the diode directly across the motor's own terminals, protecting the driver stage (in this project's case, the L298N's output transistors) from the motor's own back-EMF the instant power is cut or direction is reversed. It is worth noting explicitly that L298N breakout boards commonly ship with these flyback diodes already built into the board across each output, so the discrete flyback diode this report is describing is primarily relevant to any *additional* relay coil the project adds – e.g. a relay used for the master power cutoff in Section 2.3 – rather than something that needs to be added again at the motor terminals if the L298N module already includes it.

```
1 // Relay driven from a GPIO through a transistor stage; the flyback
2 // diode is wired in hardware directly across the relay COIL
3 // terminals (cathode to the coil's +12V side, anode to the
4 // switched/transistor side) -- not shown here since it is a wiring
5 // detail, not a firmware one.
6 const int relayControlPin = 25;
7
8 void setup() {
9     pinMode(relayControlPin, OUTPUT);
10    digitalWrite(relayControlPin, LOW); // motor supply cut on boot
11 }
```

Listing 1: Flyback diode placement is a wiring decision, not firmware, but this snippet is provided for context to show the discrete relay coil this diode protects when the ESP32 drives a relay for a master power cutoff

3.2 Optocoupler Isolation Boards

The optocoupler board is the main electrical barrier between the ESP32's GPIOs and everything on the motor side of the project. As established in Section 2.1, an optocoupler provides true galvanic isolation – no shared electrical path – between its input and output halves, using light rather than a wire to carry the signal across. Placed between the ESP32's GPIOs and the L298N's logic inputs, it means that a fault anywhere downstream (a shorted L298N input, a miswired motor lead, static discharge picked up through the motor's cabling) has no direct conductive path back to the GPIO pin, only an optical one that cannot conduct a damaging current or voltage. This is a materially different kind of protection from the flyback diode in Section 3.1, which absorbs an expected transient on a specific line; the optocoupler instead removes the electrical connection itself, which is why it is described in this report as the isolation *boundary* rather than a component that reacts to a specific fault.

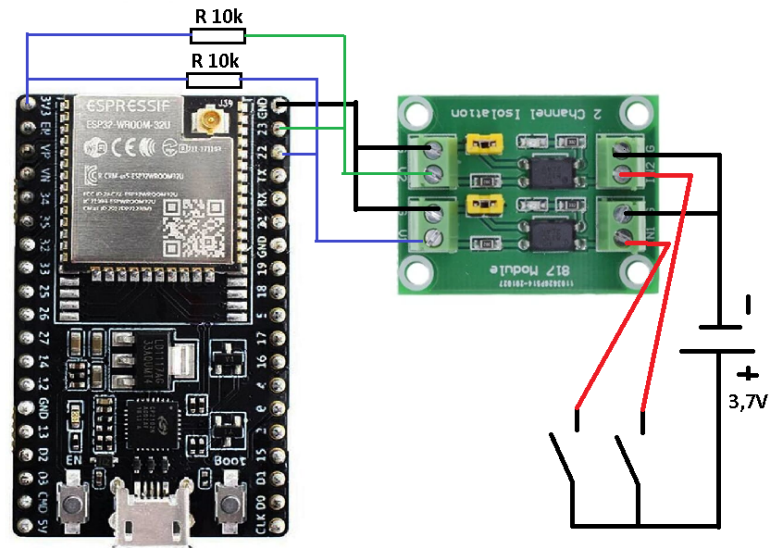


Figure 5: An ESP32 wired to a 2-channel PC817 optocoupler isolation board: the ESP32's GPIOs (through current-limiting resistors) drive the input side of each channel, while the output side is switched independently on its own isolated 3.7V supply – concretely showing the galvanic separation described above, with no shared electrical path between the two sides of the board.

3.3 Power Supply Isolation

The **ESP32 should not share a power rail with the motor, because the motor's current draw is neither steady nor small relative to what the ESP32 needs**. A motor at startup can draw five to fifteen times its steady running current for the first tens of milliseconds as it transitions from stalled to spinning, and if that surge is drawn from the same supply rail feeding the ESP32, the resulting voltage sag can be severe enough to momentarily drop the rail voltage far enough that the ESP32 interprets the dip as a brownout condition and resets. Motors also put continuous electrical noise onto whatever supply line they share, independent of the startup surge, which is a second, ongoing reason to keep the two loads apart rather than a one-time startup problem that a large enough capacitor could fully solve. The practical fix used throughout this report's reference material is the same one already applied in this project's own L298N wiring: the motor is powered from its own external supply, separate from the ESP32's own 3.3V/5V logic rail, with the two power domains tied together only at a single common ground point – never at both the positive supply *and* ground, which would recreate the shared-rail problem through a different path.

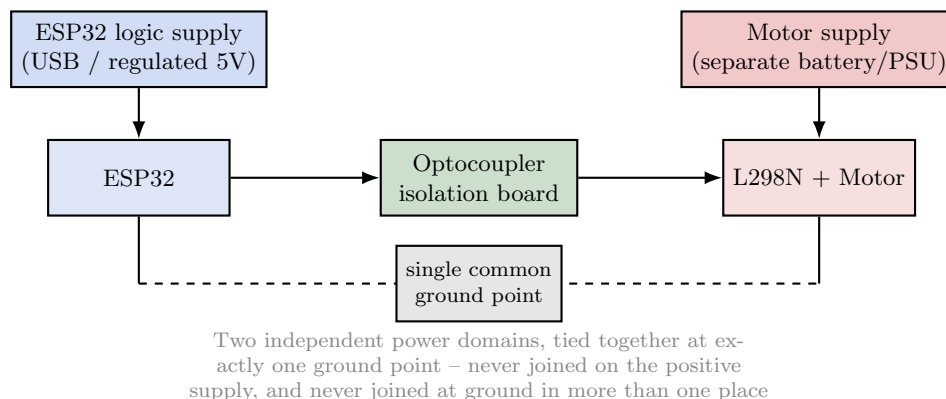


Figure 6: Recommended power architecture: the ESP32 and the motor/L298N draw from separate supplies, cross only through the optocoupler's signal path, and share ground at a single point.

4. EMI – The Practical Minimum

Industrial motor-drive EMI guidance runs to shielded cable, ferrite cores wound at precise angles, dedicated AC line filters, and dual-ended shield termination on communication buses. Almost none of that is proportionate to a single brushed DC motor and one or two relays on a breadboard-scale project. This section deliberately narrows the scope to what actually matters at that scale.

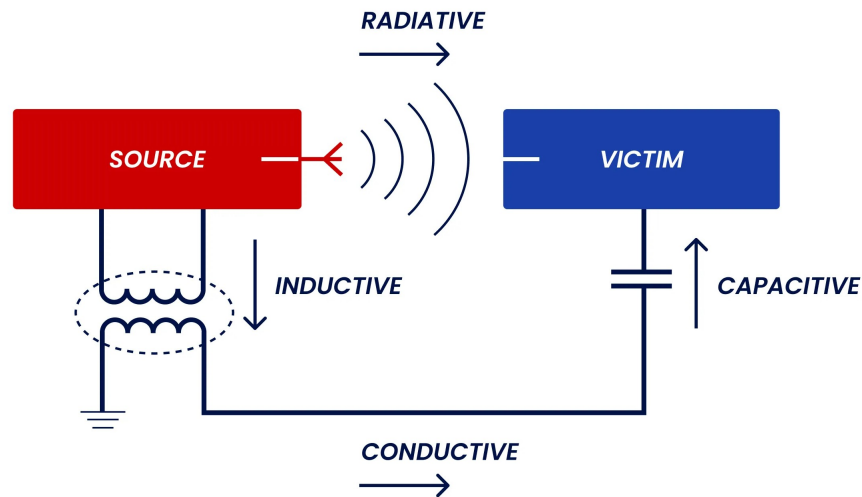


Figure 7: The four ways EMI couples from a source to a victim circuit

4.1 What Causes EMI in This Setup

Brush arcing is the motor's own, continuous EMI source. A brushed DC motor's commutator physically breaks and re-makes electrical contact with each brush many times per revolution; every one of those make/break transitions is a small spark, and optimizing that commutation process to reduce sparking is one of the direct ways to reduce the high-frequency noise a brushed motor generates, though the sparking itself cannot be eliminated by wiring choices alone – it is a property of how the motor is built.

Relay and contactor switching is the second, intermittent EMI source. Every time a relay's mechanical contacts open under load, the current flowing through the contact briefly arcs across the shrinking air gap before fully breaking, producing a burst of broadband noise similar in character to brush arcing but happening only at the moment of switching, rather than continuously during motor rotation.

4.2 The Three Mitigations That Matter at This Scale

Single-point grounding. Establishing a single, low-impedance ground path eliminates ground loops and reduces circulating currents that would otherwise let noise generated on the motor side re-enter the signal side through a second, unintended ground connection. This is the same principle already applied in Section 3.3's power architecture – the ESP32 and motor domains share ground at exactly one point – and is worth restating here specifically as an EMI mitigation, not just a brownout-prevention one, because a ground *loop* (two or more paths between the same two ground points) is precisely what lets circulating noise current flow where it otherwise would not.

Twisted-pair signal wiring. Using twisted-pair cabling on signal and DC power lines helps cancel out noise that would otherwise couple onto a single conductor: because the two conductors of a twisted pair run physically close together and reverse position at every twist, noise picked up inductively from a nearby motor cable induces an almost-equal, opposite-sign voltage in each conductor, which the differential signal at the receiving end largely

cancels out. At this project's scale, this applies most directly to the ENA/IN1–IN4 wiring between the ESP32 (or the optocoupler board) and the L298N, and to any relay coil control wiring, rather than to the short, direct motor-terminal-to-L298N connections.

Keeping signal cables away from motor cables. Motor cables should be kept away from other signal and communication lines, and where the two must cross, routing them at right angles rather than running them in parallel is preferable, because parallel runs maximize the length over which inductive and capacitive coupling can occur, while a perpendicular crossing minimizes it. On a breadboard or small enclosure, this is a physical layout decision – routing the motor's power leads along one side of the build and the ESP32's signal wiring along the other – rather than a component to add.

Interesting detail found during research: full industrial guidance draws a sharp distinction between how a shield should be grounded depending on what it is protecting – a sensitive low-level sensor cable's shield should be terminated at only one end to avoid a ground loop, while a communication bus like CAN or RS485 should have its shield terminated at both ends for continuous low-impedance grounding against high-frequency coupling. This project uses neither shielded cable nor a communication bus, so the distinction is not directly applicable, but it is a useful signpost for the next mitigation to reach for if twisted pair and physical separation alone turn out not to be enough: shielding, and specifically *which* shield-grounding scheme, would be the next lever, not simply “add more shielding.”

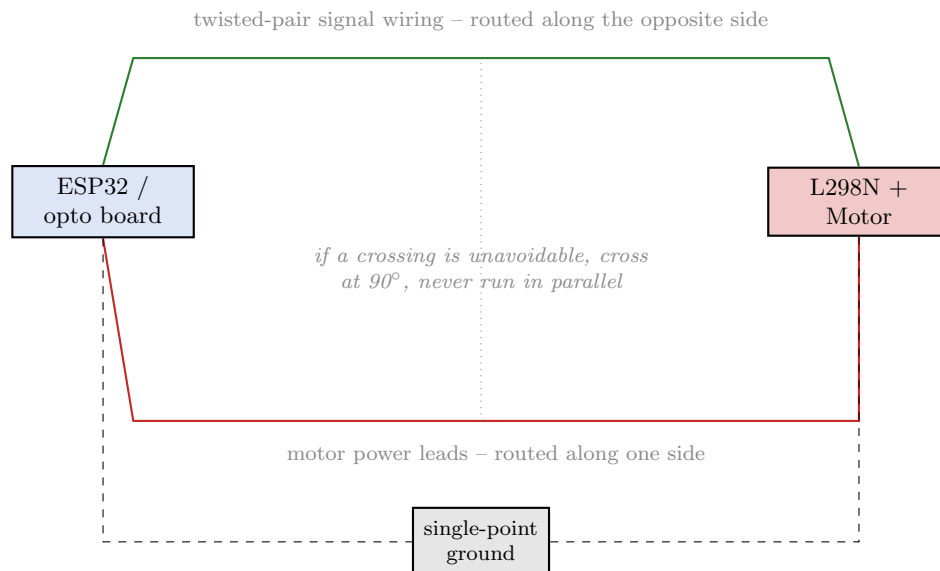


Figure 8: The three EMI mitigations applied to this project's physical layout: motor and signal cables kept on opposite sides of the build, signal wiring twisted, and a single ground point tying the two domains together.

5. Summary & Checklist

Section 2 established that a relay, an optocoupler, and an SSR are not interchangeable: the L298N's logic-level, PWM-carrying ENA/IN1–IN4 lines call for an optocoupler's signal isolation, while cutting the motor's own high-current supply line entirely – something the L298N's own H-bridge is not meant to do – calls for a relay or an SSR instead. Section 3 separated three distinct ESP32 protections that are often lumped together: a flyback diode absorbs a specific, expected transient on an inductive load (most relevant here to any added relay coil, since the L298N module typically already includes flyback diodes at the motor terminals); an optocoupler isolation board removes the electrical connection between the ESP32 and the motor side entirely; and a separate power supply, tied to the ESP32's supply at a single ground point, prevents the motor's startup surge and continuous noise from ever reaching the ESP32's own rail in the first place. Section 4 narrowed industrial-scale EMI guidance down to the three mitigations proportionate to this project's scale – single-point grounding, twisted-pair signal wiring, and physical separation between motor and signal cabling – while flagging shielded cable and communication-bus-style dual-ended shield grounding as the next lever if those three prove insufficient, rather than something to reach for immediately. Taken together, the checklist for the next motor-driven build is: isolate every logic line to the driver through an optocoupler; put a flyback diode directly across any relay coil added beyond the driver module itself; power the motor from its own supply, joined to the logic supply at exactly one ground point; and keep motor and signal wiring twisted and physically separated before reaching for anything more elaborate.

6. References

1. embeddeddesignblog, *Opto Coupler vs SSR vs Mechanical Relay*. <https://embeddeddesignblog.blogspot.com/2021/04/opto-coupler-vs-ssr-vs-mechanical-relay.html>
2. GEYA, *Optocoupler vs Solid State Relay: Key Differences, and How to Choose*. <https://www.geya.net/optocoupler-vs-solid-state-relay/>
3. USAC, *Understanding the Difference of Optocouplers vs Relays*. <https://usac123.com/blog/optocoupler-vs-relay>
4. Onesto Electronic Parts, *Guide to Choosing Optocouplers, Relays, and SSRs*. <https://www.onesto-ep.com/blog/guide-to-choosing-optocouplers-relays-and-ssrs/>
5. Huyu Electric, *Optocoupler vs Relay vs SSR: How to Choose the Right Component for Your Design*. <https://huyuelectric.com/optocoupler-vs-relay-vs-ssr/>
6. VIOX, *Stop Relay Failures: The Engineer's Guide to Choosing Between Optocouplers, Relays, and SSRs*. <https://viox.com/stop-relay-failures-the-engineers-guide-to-choosing-between-optocouplers-relays-and-ssrs/>
7. HUIMU Electronics, *What is the Solid State Relay (SSR)?*. <https://electronics.huimultd.com/Blog/what-is-the-solid-state-relay/>
8. Components101, *Safeguarding Circuits: The Essential Guide to Flyback Diodes*. <https://components101.com/articles/safeguarding-circuits-essential-guides-to-flyback-diodes>
9. IC Components, *Choosing and Using the Right Flyback Diode*. <https://www.ic-components.com/blog/flyback-diodes-from-basics-to-advanced-uses.jsp>
10. Industrial Monitor Direct, *Flyback Diodes: Protecting Relay Coils from Back EMF*. <https://industrialmonitordirect.com/blogs/knowledgebase/flyback-diodes-protecting-relay-coils-from-back-emf>

11. Altium, *Using Flyback Diodes in Relays Prevents Electrical Noise in Your Circuits*. <https://resources.altium.com/p/using-flyback-diodes-relays-prevents-electrical-noise-your-circuits>
12. UltraLibrarian, *Choosing a Flyback Diode: Selection Process*. <https://www.ultralibrarian.com/2026/1/12/choosing-a-flyback-diode-selection-process-ulc>
13. Cadence PCB Design Blog, *Selecting Flyback Diodes for 5V Relay Coil Suppression*. <https://resources.pcb.cadence.com/blog/2022-selecting-flyback-diodes-for-5v-relay-coil-suppression>
14. MicrocontrollersLab, *Interface L298N DC Motor Driver Module with ESP32 using Arduino IDE*. <https://microcontrollerslab.com/l298n-dc-motor-driver-module-esp32-tutorial/>
15. IoT Web Planet, *ESP32 With L298N DC Motor Driver*. <https://iotwebplanet.com/esp32-with-l298n-dc-motor-driver/>
16. Last Minute Engineers, *How to Control DC Motors Using ESP32 and L298N Module*. <https://lastminuteengineers.com/esp32-l298n-dc-motor-control-tutorial/>
17. All About Circuits Forum, *Need help wiring NPN proximity sensor → Optocoupler → ESP32 → L298N Motor driver*. <https://forum.allaboutcircuits.com/threads/need-help-wiring-npn-proximity-sensor-optocoupler-esp32-l298n-motor-driver-avoid-frying-esp32.208834/>
18. Dynamic Repair & Electric, *Mitigation of EMI, EMC, & EMS for Industrial Machine Builders*. <https://www.dynamicrep.com/mitigation-of-emi-emc-ems-for-industrial-machine-builders>
19. AutomationDirect, *Applied EMI/RFI Techniques*. https://support.automationdirect.com/docs/emi_mitigation.pdf
20. Roboteq, *Understanding EMI: Causes, Effects, and Practical Mitigation Strategies*. <https://www.roboteq.com/technology/all-blogs/581-understanding-emi>
21. Lunyee, *DC Motor EMI Analysis*. <https://www.lunyee.com/news/industry/dc-motor-emi-analysis.html>
22. DroneBot Workshop Forums, *Esp32cam brownout detector triggered*. <https://forum.dronebotworkshop.com/esp32-esp8266/re-esp32cam-brownout-detector-triggered/>
23. espboards.dev, *ESP32 Power Drop Under Load*. <https://www.espboards.dev/troubleshooting/issues/power/esp32-power-drop-under-load/>
24. Techietory, *Why Do Robots Need Separate Power for Motors and Logic? Complete Guide*. <https://techie-tory.com/robotics/why-robots-need-separate-power-motors-logic/>
25. espboards.dev, *ESP32 External Power Guide*. <https://www.espboards.dev/troubleshooting/issues/power/esp32-external-power-guide/>